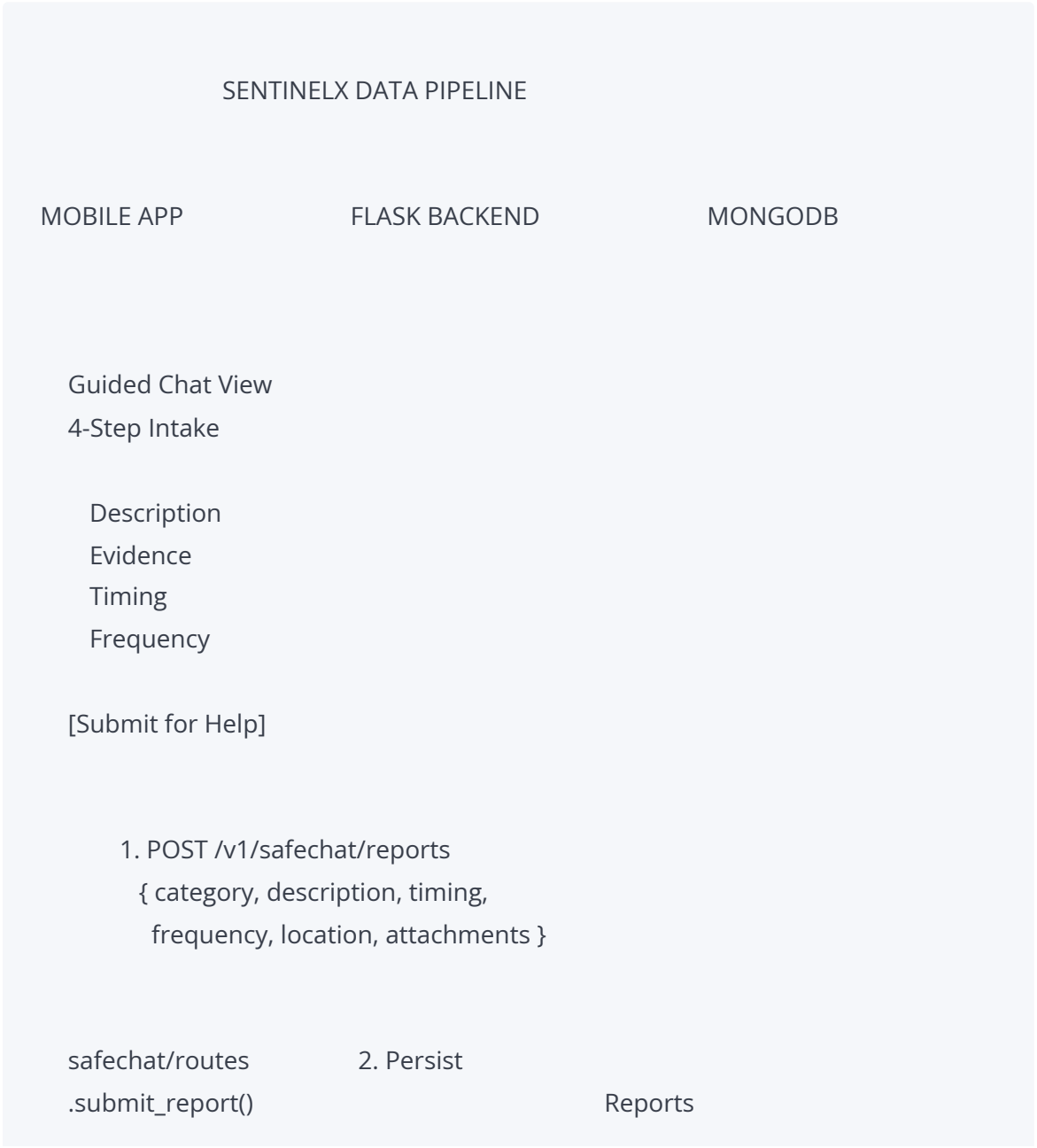
SentinelX — Technical Artifact: API Workflow & Communication Matrix

---

1. End-to-End Dynamic Workflow Layout

The following ASCII diagram illustrates the complete 7-step telemetry path from mobile app intake through human-gated AI analysis and real-time dashboard update:



Collection

Agency proximity routing via get_nearest_agencies(lat, lng, limit=4)

status: "pending_analysis"

3. Socket.IO "new_report"
room: "agency_{id}"

DASHBOARD UI
(Web Client)

Alert card appears in Reports tab with badge

[Analyst clicks Analyze]

4. POST /agency/reports/<id>/analyze
(jwt_required analyst identity verified)

besafe_app.py
.agency_analyze_report()

5. Dual Model Pipeline

Bi-LSTM (ONNX) model_pipeline .predict_threat

LLM Provider (OpenAI / Gemini / FreeModel)

Threat prob. JSON structured
0.0 1.0 output via
Pydantic schema

Compound Formula:
priority = (confidence × 0.6)
+ (time_decay × 0.3)
+ (unacked × 0.1)

6. update_report_analysis() MongoDB
writes ai_Analysis + status update

7. Socket.IO "report_analyzed"
room: "agency_{agency_id}"
payload: { report_id, ai_Analysis }

DASHBOARD UI

XAI Panel
slides open
below the
detail card
(no page
refresh)

Status: "triaged"
Analyst reviews
Acknowledge
Resolve
Close

2. API Endpoint Protocol Directory

Endpoint 1: Submit Guided Field Report

Route: `POST /v1/safechat/reports`

Blueprint: `safechat_bp` — `safechat/routes.py:submit_report()`

Authentication: `@require_auth` — Bearer JWT access token in Authorization header.

Validation rules (enforced server-side):

Field	Type	Constraints
`category`	string	Must be one of: `abuse-home`, `harassment`, `unsafe-ride`, `threats`, `other`
`description`	string	Required, must not be empty
`timing`	string	Must be one of: `just-now`, `today`, `this-week`, `longer-ago`
`frequency`	string	Must be one of: `first`, `few`, `many`
`submitForHelp`	boolean	If `true`, report is agency-routed; if `false`, stored as private
`location`	object	Optional `{ lat, lng }` — used for nearest-agency routing
`attachments`	array	Optional — each entry must be a dict with `{ id, type, uri, url, name, mimeType, size, createdAt }`

Request:

```
POST /v1/safechat/reports
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
Content-Type: application/json
```

```
{
  "category": "harassment",
  "description": "Received repeated threatening messages after rejecting unwanted adva
  "timing": "this-week",
```

```
  "frequency": "few",
  "submitForHelp": true,
  "location": {
    "lat": 15.5007,
    "lng": 32.5599
  },
  "attachments": [
    {
      "id": "a1b2c3d4",
      "type": "photo",
      "uri": "https://res.cloudinary.com/.../besafe/evidence/photo_abc123.jpg",
      "url": "https://res.cloudinary.com/.../besafe/evidence/photo_abc123.jpg",
      "name": "Screenshot_of_threat.jpg",
      "mimeType": "image/jpeg",
      "size": 245760,
      "createdAt": "2026-06-21T10:30:00Z"
    }
  ]
}
```

Response (201 Created):

```
{
  "success": true,
  "data": {
    "message": "Report saved",
    "reportId": "667b8c1f2a3b4c5d6e7f8a9b"
  }
}
```

Post-submission side effects:

- If `submitForHelp` is `true` and an agency was assigned, `socketio.emit("new_report", serialize_report(saved), room="agency_{id}")` pushes the serialized report to the dashboard in real time.
- Agency routing uses `get_nearest_agents(lat, lng, limit=4)` for location-aware dispatch, falling back to `get_all_agencies()` if no agency has a geotagged headquarters.

Endpoint 2: Execute Human-Gated AI Analysis

Route: `POST /agency/reports/<report_id>/analyze`

Handler: `besafe_app.py:agency_analyze_report()`

Authentication: `@jwt_required()` — Flask-JWT-Extended. The `agency_id` extracted from the JWT identity must match the document's `assignedAgencyId`.

Request:

```
POST /agency/reports/667b8c1f2a3b4c5d6e7f8a9b/analyze
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
Content-Type: application/json
```

Body is empty — the report's four intake fields are read from the MongoDB document.

Response (200 OK) — Successful analysis:


```
{
  "success": true,
  "analysis": {
    "identified_pattern_type": "Persistent Behavioral Escalation",
    "severity_rating": 0.82,
    "pattern_tags": ["COERCION", "DIGITAL_EXPLOITATION"],
    "escalation_risk": "HIGH",
    "timeline_urgency": "IMMEDIATE",
    "isolation_risk_detected": true,
    "investigative_priority": "CRITICAL",
    "explainable_ai_report": "Subject exhibits a repetitive pattern of digital harassment wi
    "provider": "openai"
  }
}
```


Response (500 Error — Pipeline failure):


```
{
  "error": "AI analysis failed",
  "detail": "[provider=openai] 429 Too Many Requests"
}
```

Error states returned:

Status	Condition
404	Report not found, or `assignedAgencyId` does not match the JWT identity
500	LLM returns `identified_pattern_type: "Pipeline_Error"` — see `error_message` for provider-specific details

Post-analysis side effects:

- `update_report_analysis(report_id, analysis)` writes `ai_Analysis` and sets `status` to `"triaged"` in MongoDB.
- `socketio.emit("report_analyzed", { report_id, ai_Analysis }, room="agency_{id}")` pushes the analysis payload to the dashboard.

Endpoint 3: Real-Time Ambient Signal Intake

Route: `POST /v1/safety/analyze`

Blueprint: `safety_bp` — `safety/routes.py:analyze()`

Authentication: `@require_auth` — Bearer JWT access token.

Purpose: Processes ambient speech transcription from the mobile app's microphone listener. Runs the text through the ONNX-hosted Bi-LSTM model and returns a threat verdict. The mobile app uses this to decide whether to trigger the threat-detection toast and push notification.

Request:

```
POST /v1/safety/analyze
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
Content-Type: application/json
```

```
{
  "text": "He said if I tell anyone he will find me and hurt my family. I don't know what to
}
```

Response (200 OK):

```
{
  "success": true,
  "data": {
    "prediction": "Threat",
    "confidence": 0.87,
    "model_version": "1.0.00",
    "shouldTriggerSOS": true
  }
}
```

Threshold logic (`safety_service.py`):

```
THREAT_THRESHOLD = 0.75
should_trigger_sos = label.lower() == "threat" and confidence >= THREAT_THRESHOLD
```

The `shouldTriggerSOS` boolean is the only value the mobile app reads to decide whether to show the threat-detected toast. No autonomous SOS dispatch occurs — the user must press the SOS button.

Response fields:

Field	Type	Description
`prediction`	string	`"Threat"` or `"Non-Threat"`
`confidence`	float	Bi-LSTM sigmoid output, 0.0–1.0
`model_version`	string	Semantic version of the deployed ONNX model
`shouldTriggerSOS`	boolean	`true` only when prediction is Threat **and** confidence ≥ 0.75

Error states:

Status	Condition
400	`text` field is missing or empty
500	Bi-LSTM model call returns `None` (service unavailable)

---

3. Real-Time WebSockets Event Lifecycle (Socket.IO)

Socket.IO is configured with transports `['websocket', 'polling']` for maximum compatibility. The server multiplexes events into named rooms scoped by entity type (`agency_{id}` , `user_{id}`).

Event: `new_report`

Property	Value
Direction	Server Web Client (dashboard)
Trigger	`POST /v1/safechat/reports` completes with `submitForHelp: true` and a matching agency found via nearest-agency routing
Emission code	`socketio.emit("new_report", serialize_report(saved), room=f"agency_{agency_id}")`
Dashboard handler	`dashboard.js:1002-1010` — adds to local `reports` map, re-renders overview stats + report list, plays toast + alert sound

Payload (extracted from `safechat/routes.py:serialize_report()`):


```
{
  "id": "667b8c1f2a3b4c5d6e7f8a9b",
  "userId": "65a1b2c3d4e5f6a7b8c9d0e1",
  "category": "harassment",
  "description": "Received repeated threatening messages...",
  "timing": "this-week",
  "frequency": "few",
  "location": { "lat": 15.5007, "lng": 32.5599 },
  "status": "pending_analysis",
  "priority": "high",
  "submittedToAgency": true,
  "assignedAgencyId": "60a1b2c3d4e5f6a7b8c9d0e2",
  "attachments": [
    {
      "id": "a1b2c3d4",
      "type": "photo",
      "uri": "https://res.cloudinary.com/...",
      "url": "https://res.cloudinary.com/...",
      "name": "Screenshot_of_threat.jpg",
      "mimeType": "image/jpeg",
```

```
      "size": 245760,
      "createdAt": null
    }
  ],
  "ai_Analysis": null,
  "createdAt": "2026-06-21T10:31:00",
  "updatedAt": "2026-06-21T10:31:00"
}
```

UI effect: Report card appears in the Reports tab with a `pending_analysis` badge. The overview page stat increments. A toast notification slides in from the top-right corner with the category label and priority. An alert chime plays.

Event: `report_analyzed`

Property	Value
Direction	Server Web Client (dashboard)
Trigger	`POST /agency/reports/<report_id>/analyze` completes successfully after the LLM returns a valid structured JSON result
Emission code	`socketio.emit("report_analyzed", { report_id, ai_Analysis: analysis }, room=f"agency_{agency_id}")`
Dashboard handler	`dashboard.js:1011-1019` — patches `reports[id].ai_Analysis`, sets `status = 'triaged'`, re-renders the detail panel if currently open, re-renders the report list, shows a toast

Payload:

```
{
  "report_id": "667b8c1f2a3b4c5d6e7f8a9b",
  "ai_Analysis": {
    "identified_pattern_type": "Persistent Behavioral Escalation",
    "severity_rating": 0.82,
    "pattern_tags": ["COERCION", "DIGITAL_EXPLOITATION"],
    "escalation_risk": "HIGH",
    "timeline_urgency": "IMMEDIATE",
    "isolation_risk_detected": true,
    "investigative_priority": "CRITICAL",
```

```
    "explainable_ai_report": "Subject exhibits a repetitive pattern...",
    "provider": "openai"
  }
}
```

UI effect — XAI Justification Panel:

The dashboard's `renderReportDetail()` function checks `isAnalyzed = r.ai_Analysis && r.status !== 'pending_analysis'`. When the `report_analyzed` socket event fires and the local state is patched, the next render includes the full XAI panel:

```
[AI] Threat Analysis                    .xai-panel with severity-colored
                                        left border

Sever-      Pattern:
ity         Persistent
Score       Behavioral
            Escalation
82%

Esc-        Investigation
ala-        Priority:
tion        CRITICAL
HIGH

[COERCION] [DIGITAL_                    .tag-filled pills
EXPLOITATION]

Subject exhibits a repetitive           .xai-report (explainable AI)
pattern of digital harassment
with escalating frequency...

[Acknowledge] [Resolve] [Close]         action buttons enabled
```

The panel uses staggered CSS animations (`.anim-section` with `animation-delay: 0s, 0.05s, 0.1s, 0.15s, 0.2s`) so each section fades in sequentially — no page refresh required.

Supporting Events

Event	Direction	Trigger	Payload
`new_alert`	Server Dashboard	SOS button press creates an alert via `_route_to_agency()`	`{ id, user_id, user_name, user_phone, user_photo, transcribed_text, confidence, gps_lat, gps_lng, status, agency_id, sos_contacts, created_at }`
`alert_analyzed`	Server Dashboard	`POST /alerts/<id>/analyze` completes	`{ alert_id, ai_analysis: {...} }`
`alert_status_update`	Server Dashboard	Agency acknowledges/resolves an alert, or user presses “I’m Safe”	`{ alert_id, status }`
`report_status_update`	Server Dashboard	Agency reviews/resolves/closes a report	`{ report_id, status }`
`location_update`	Client Server Dashboard	Mobile app streams GPS during active safety check	`{ alert_id, lat, lng }`

Connection lifecycle

1. **Dashboard** calls `io(BASE_URL, { transports: ['websocket', 'polling'] })` on page load.
2. On `connect` , the client emits `join` with `{ agency_id }` .

3. Server calls `join_room(f"agency_{agency_id}")` — all subsequent emits to that room reach this dashboard instance.
4. On `disconnect`, the client automatically reconnects via the polling fallback; Socket.IO's built-in retry handles transient network loss.